



UNITED INTERNATIONAL UNIVERSITY  
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## PROJECT PROPOSAL

# HaluRISC

Calibrated, Explainable Hallucination  
Risk Prediction for LLM Outputs

---

Submitted By

|                           |            |
|---------------------------|------------|
| <b>Saiful Alam Sabbir</b> | 0112320105 |
| <b>Md. Atikur Rahaman</b> | 0112310298 |
| <b>MD. Miraz Ahamed</b>   | 0112310524 |

Supervision

**Ohidujjaman Tuhin, PhD**

Associate Professor, Dept. of Computer Science and Engineering  
United International University

Course: Machine Learning, Section: E

# 1 Project Overview

Large language models (LLMs) like ChatGPT are very good at writing text that sounds confident and correct, but they sometimes generate information that is wrong or made up — this is called a hallucination. This project builds a complete system that predicts how likely an LLM answer is to be hallucinated. The system is black-box friendly — it only needs the question, the answer, and (optionally) some context, without ever looking inside the LLM. It is lightweight — it uses simple text features and classical machine learning (XGBoost), so it runs in milliseconds on a normal computer, with no GPU and no API fees. It is explainable — for every prediction it shows *why* the answer looks risky (for example: “the answer contradicts the provided context”, or “the answer contains numbers that are not supported”). Finally, it is presentable — a polished, interactive web dashboard lets a judge or supervisor test the system live, type any question and answer, and see the risk score and explanation instantly. The novelty is not in inventing hallucination detection, but in combining NLI-based consistency features, calibrated risk scores, and tested explanations into one lightweight system — a combination no published work offers as a single product.

## 2 Motivation

LLMs are now used in real products — chatbots, Q&A systems, content review tools — where a wrong answer has a real cost, and the most common way to detect hallucinations is to ask the LLM itself to judge its own output [2], which is slow, expensive, and sometimes wrong. A lightweight model trained on simple text features (word overlap, named entities, contradiction with context, numbers, hedging words) can flag risky answers almost instantly and at almost zero cost. A risk score is also more useful than a simple “yes/no” flag, because real systems can decide what to do with a risky answer — show a warning, ask for human review, or fetch better evidence. And because the final result is a working product with a clean UI rather than just a report, the project is fully ready for live demonstration.

## 3 Objectives

Research question: Can a lightweight machine learning model predict hallucination risk in LLM answers using simple text features, while giving calibrated risk scores and clear explanations? To answer this, the project has five objectives: (1) build a feature extraction pipeline (about 20–30 features) that captures how well an answer agrees with the question and context — word overlap, named entities, contradiction signals, numbers, hedging words, and semantic similarity; (2) train and honestly compare classical models — a simple heuristic baseline, Logistic Regression, Random Forest, and XGBoost; (3) calibrate the risk scores so that a score of 0.8 truly means “about 80% risk”, making the score trustworthy for real use; (4) explain predictions with SHAP, giving a global view of which features matter most plus detailed case studies of individual predictions; and (5) build a polished web dashboard (React + FastAPI) with strong UI/UX — live prediction, risk gauge, explanation panel, and a demo gallery — designed for presentation and judging.

## 4 Related Work

Hallucination detection is an active research area. The most common approach asks the LLM itself to judge its own answer, as in SelfCheckGPT [2], but this is slow and expensive. To cut this cost, small language models have been used as detectors [9], and lightweight evaluation models such as Luna [10] show that efficient detection is possible in practice. Other approaches train neural classifiers on frozen language encoders [7], or combine explainable text signals with machine learning [6, 8].

On the benchmark side, HaluEval [1] and TruthfulQA [3] provide standard test beds, and fine-grained methods such as FActScore [4] evaluate factual precision in detail. Newer benchmarks such as FaithBench [11] show that current detectors still fail on challenging cases, and SpikeScore [12] shows that cross-domain generalization remains an open problem.

None of these works combine lightweight engineered features with calibrated risk scores, tested explanations, and a deployable dashboard — this is the gap the project fills.

## 5 Methodology

The pipeline has seven clear steps:

1. Data preparation — load the HaluEval QA dataset, clean the text, and manually inspect 50 random samples to verify label quality before training.
2. Feature extraction — compute features in seven groups: length/style, lexical overlap, entity overlap, NLI contradiction (using a lightweight pre-trained NLI model), numeric consistency, hedging phrases, and semantic similarity.
3. Model training — 5-fold stratified cross-validation, hyperparameter tuning for XGBoost, and every experiment repeated with 3 random seeds, reported as mean  $\pm$  standard deviation.
4. Model comparison — heuristic baseline, Logistic Regression, Random Forest, and XGBoost, judged on F1, AUROC, precision, recall, and MCC.
5. Calibration — apply Platt scaling to the final model [5] and measure the gain with Expected Calibration Error (ECE) and Brier score.
6. Explanation — SHAP global feature importance plus 3 local case studies; error analysis on 20 wrong predictions (10 false positives, 10 false negatives).
7. Deployment — FastAPI backend serving the trained model, and a React (Vite) dashboard with a clean, modern UI: risk gauge, feature breakdown, and live input panel.

Statistical rigor: model differences are tested with McNemar’s test and bootstrap confidence intervals, so conclusions are not based on chance from a single split.

External comparison: results are compared with published HaluEval detection baselines, such as SelfCheckGPT [2], so a judge can see whether the scores are good or bad.

## 6 Dataset

We use the HaluEval benchmark [1] (EMNLP 2023) — the standard dataset for hallucination evaluation, widely used and well documented.

- Size: 35,000 samples across 4 task types (QA, dialogue, summarization, general).
- Used here: the QA subset (about 10,000 samples). Each sample has a question, an answer, a context, and a label (hallucinated or not).
- Why QA: it is the largest, cleanest, and most comparable subset, and it matches the demo scenario (question  $\rightarrow$  answer  $\rightarrow$  risk score).

One limitation to note: most HaluEval samples were generated by ChatGPT, so results will be framed as performance on the HaluEval benchmark rather than as general hallucination detection performance.

## 7 Expected Results

Based on published baselines on the same benchmark [1, 7], we expect the tuned XGBoost model to reach an F1 score of around 0.75–0.85 and an AUROC of around 0.80–0.90 on the QA subset. Success means matching or beating these baselines while keeping the system lightweight, explainable, and ready for live demonstration.

## 8 Expected Outcomes

1. A journal-style project report (methodology, experiments, results, error analysis, limitations).
2. A trained, calibrated XGBoost model with feature ablation and SHAP explanations.
3. Statistical significance tests (McNemar, bootstrap confidence intervals).
4. A polished, judge-ready web dashboard — live risk prediction with clean UI/UX.
5. Reproducible code: dataset preparation, feature extraction, training, and API.

## 9 Project Plan

The project runs in two phases: experiments in the first half, product in the second. Weeks 1–4 cover data preparation, feature extraction, model comparison, and calibration. Weeks 5–8 cover SHAP explanations and error analysis, the FastAPI backend, the React dashboard, and the final report and demo.

## References

- [1] Li, J., Cheng, X., Zhao, W.X., Nie, J.-Y., Wen, J.-R. (2023). *HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models*. EMNLP 2023.
- [2] Manakul, P., Liusie, A., Gales, M. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models*. EMNLP 2023.
- [3] Lin, S., Hilton, J., Evans, O. (2022). *TruthfulQA: Measuring How Models Mimic Human Falsehoods*. ACL 2022.
- [4] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P.W., Iyyer, M., Zettlemoyer, L., Hajishirzi, H. (2023). *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. EMNLP 2023.
- [5] Valentin, S., Fu, J., Detommaso, G., Xu, S., Zappella, G., Wang, B. (2024). *Cost-Effective Hallucination Detection for LLMs*. arXiv:2407.21424.
- [6] Sundaragiri, D., Reddy, L.M., Gunavardhan, P., Navahith, B. (2026). *Framework for Hallucination Detection in Large Language Models*. IJERT, Vol. 15, Issue 04. DOI: 10.5281/zenodo.20025987.
- [7] Yadav, S., Verma, N.K. (2026). *A Hybrid Framework for Hallucination Detection in Large Language Models*. IEEE Transactions on Artificial Intelligence. DOI: 10.1109/TAI.2026.3653354.
- [8] Haq, I., Saqib, M., Zhang, Y., Khan, I.A. (2026). *Quantifying Factual Divergence in Generative Models: SHAP-LIME Based Hallucination Score for LLMs*. Multimedia Systems, Vol. 32, Art. 146. DOI: 10.1007/s00530-025-02150-4.
- [9] Cheng, X., Li, J., Zhao, W.X., Zhang, H., Zhang, F., Zhang, D., Gai, K., Wen, J.-R. (2024). *Small Agent Can Also Rock! Empowering Small Language Models as Hallucination Detector*. EMNLP 2024.
- [10] Belyi, M., Friel, R., Shao, S., Sanyal, A. (2025). *Luna: A Lightweight Evaluation Model to Catch Language Model Hallucinations with High Accuracy and Low Cost*. COLING 2025 Industry Track, pp. 398–409.
- [11] Bao, F.S., Li, M., Qu, R., et al. (2025). *FaithBench: A Diverse Hallucination Benchmark for Summarization by Modern LLMs*. NAACL 2025 (Short Papers), pp. 448–461. DOI: 10.18653/v1/2025.naacl-short.38.
- [12] Deng, Y., Fang, Z., Li, S., Chen, L. (2026). *Beyond In-Domain Detection: SpikeScore for Cross-Domain Hallucination Detection*. ICLR 2026. arXiv:2601.19245.